实验 6 传输层实验

学号：21009102093
姓名：黄炜锴

传输层实验发送方初始序号（sequence number）记录表，请填写：

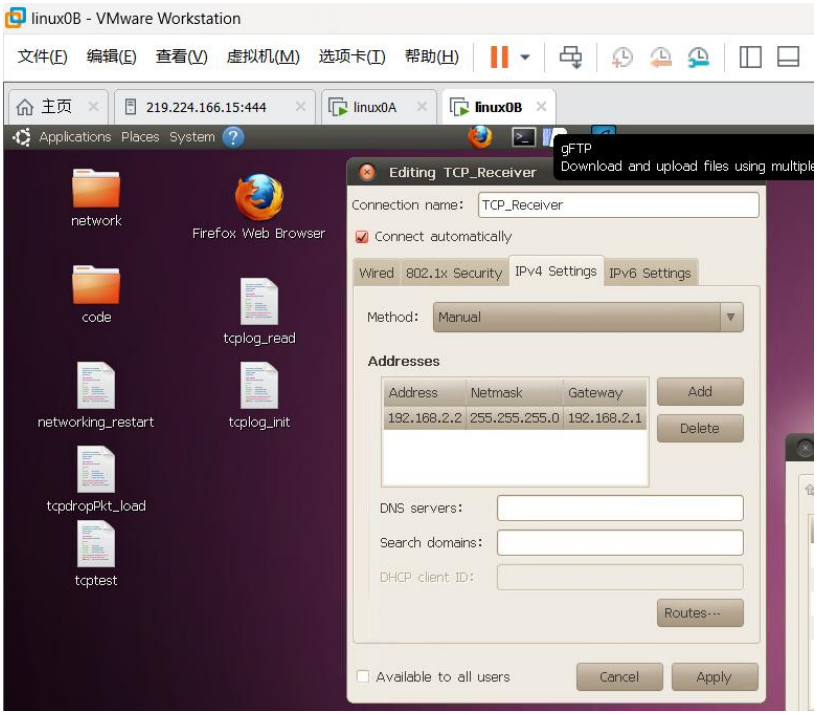
实验名称	实验班号	学号	姓名	发送方初始序号
TCP 协议基本分析	2118039	21009102093	黄炜锴	354981
滑动窗口机制和窗口侦查机制分析	2118039	21009102093	黄炜锴	354981
慢启动、拥塞避免及拥塞处理实验	2118039	21009102093	黄炜锴	354981

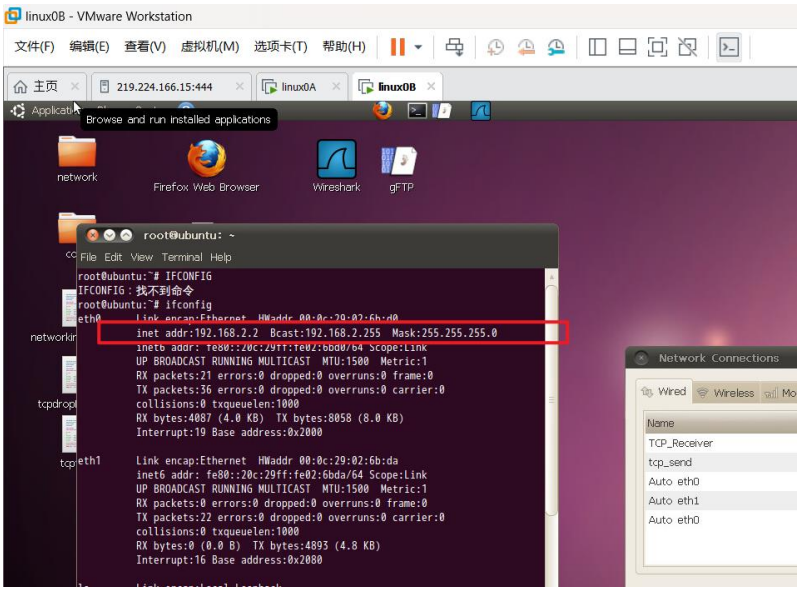
1、根据 2.6 中步骤 3 回答：TCP 的连接和建立采用的是：客户服务器方式，PCA 是客户，PCB 是服务器。先点击发送再点击接收，会出现什么问题？为什么？

会出现系统 IO 错误。因为服务器 B 没有点击接收，即没有打开 TCP 端口。

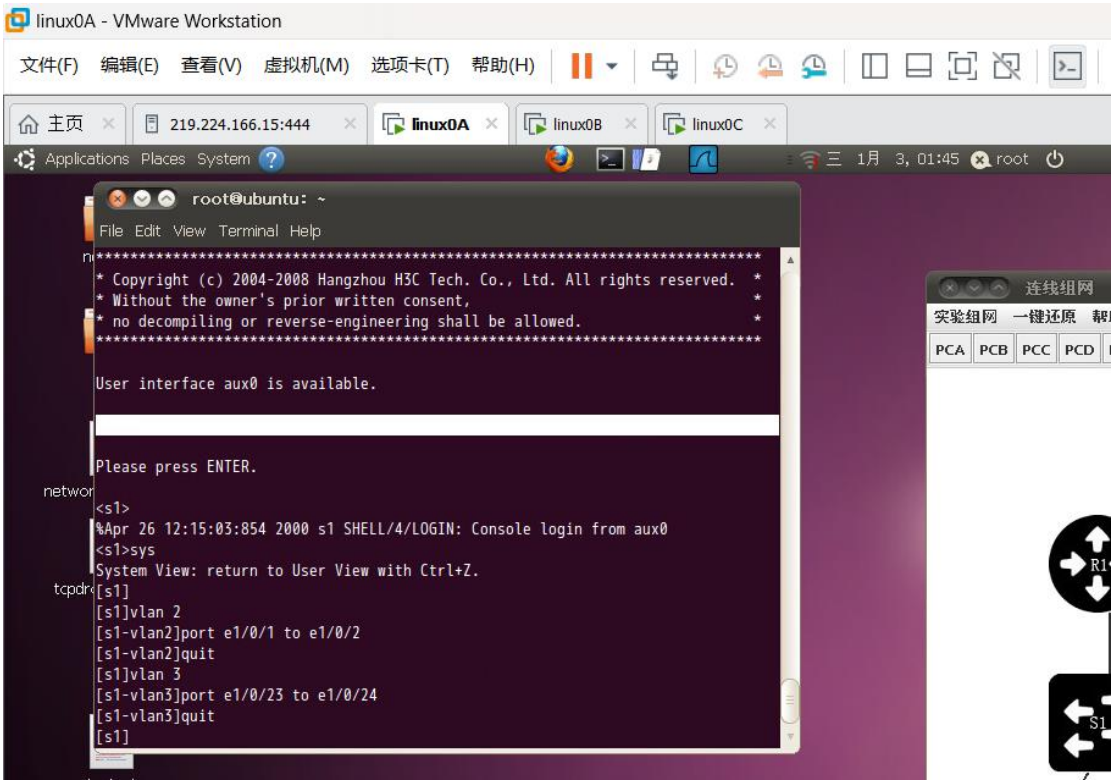
实验过程如下：

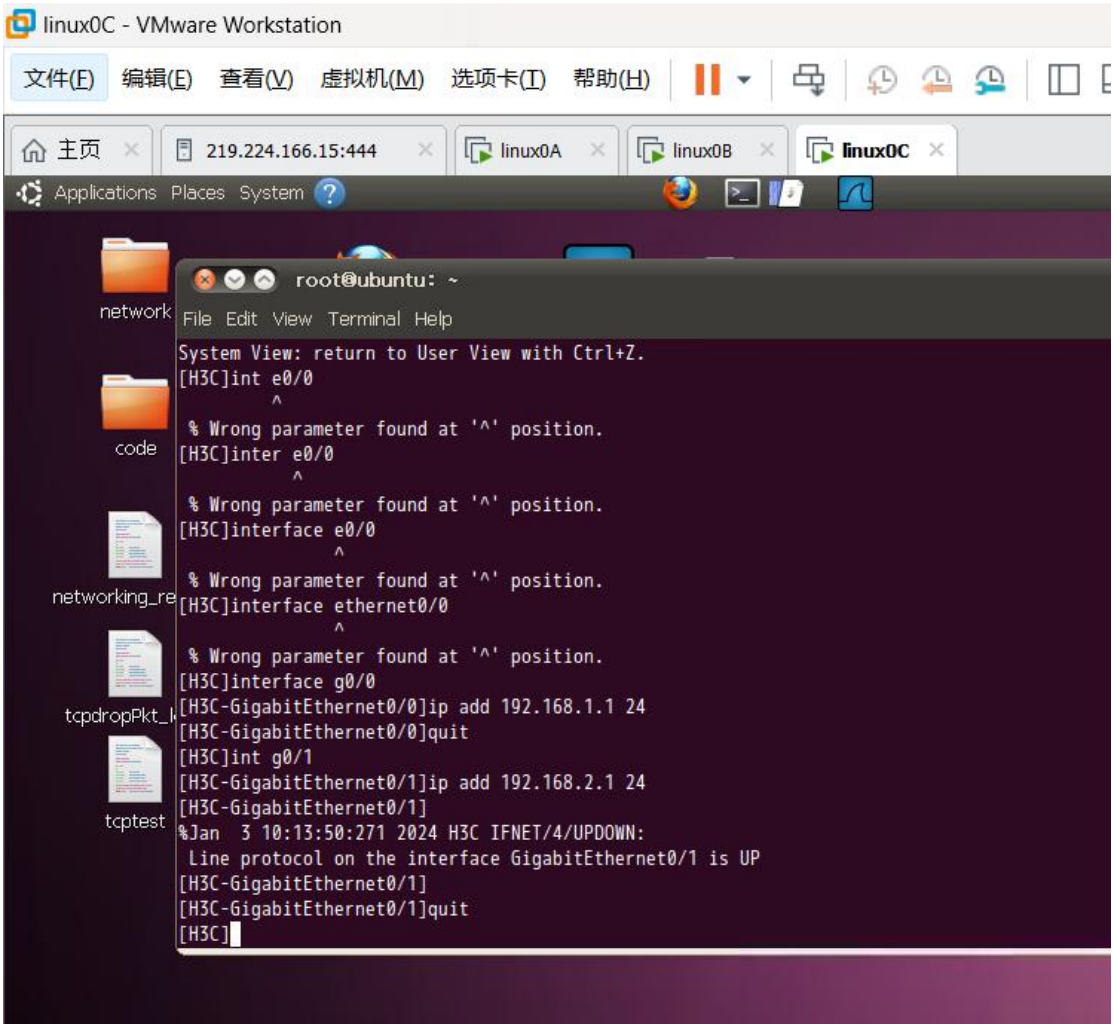
首先配置 IP 地址

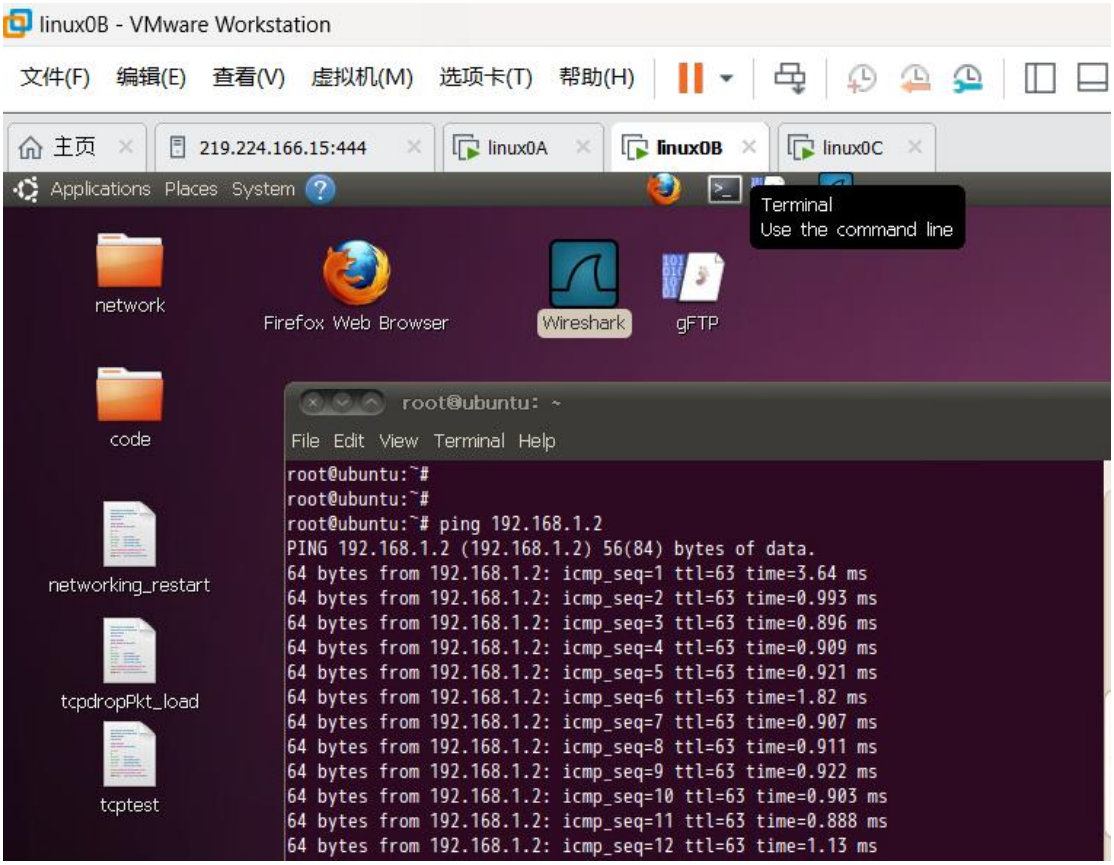



linux0B - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
主页
219.224.166.15:444
linux0A
linux0B
Browse and run installed applications
network
Firefox Web Browser
Wireshark
gFTP
root@ubuntu: ~
File Edit View Terminal Help
root@ubuntu:~# IFCONFIG
IFCONFIG：找不到命令
root@ubuntu:~# ifconfig
eth0 Link encap:Ethernet HWaddr 00:0c:29:02:6b:d0
inet addr:192.168.2.2 Bcast:192.168.2.255 Mask:255.255.255.0
inet6 addr: fe80::20c:29ff:fe02:6bd0/64 Scope:Link
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:21 errors:0 dropped:0 overruns:0 frame:0
TX packets:36 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:4087 (4.0 KB) TX bytes:8058 (8.0 KB)
Interrupt:19 Base address:0x2000
eth1 Link encap:Ethernet HWaddr 00:0c:29:02:6b:da
inet6 addr: fe80::20c:29ff:fe02:6bda/64 Scope:Link
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:22 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:0 (0.0 B) TX bytes:4893 (4.8 KB)
Interrupt:16 Base address:0x2080
Network Connections
Wired
Wireless
Name
TCP_Receiver
tcp_send
Auto eth0
Auto eth1
Auto eth0

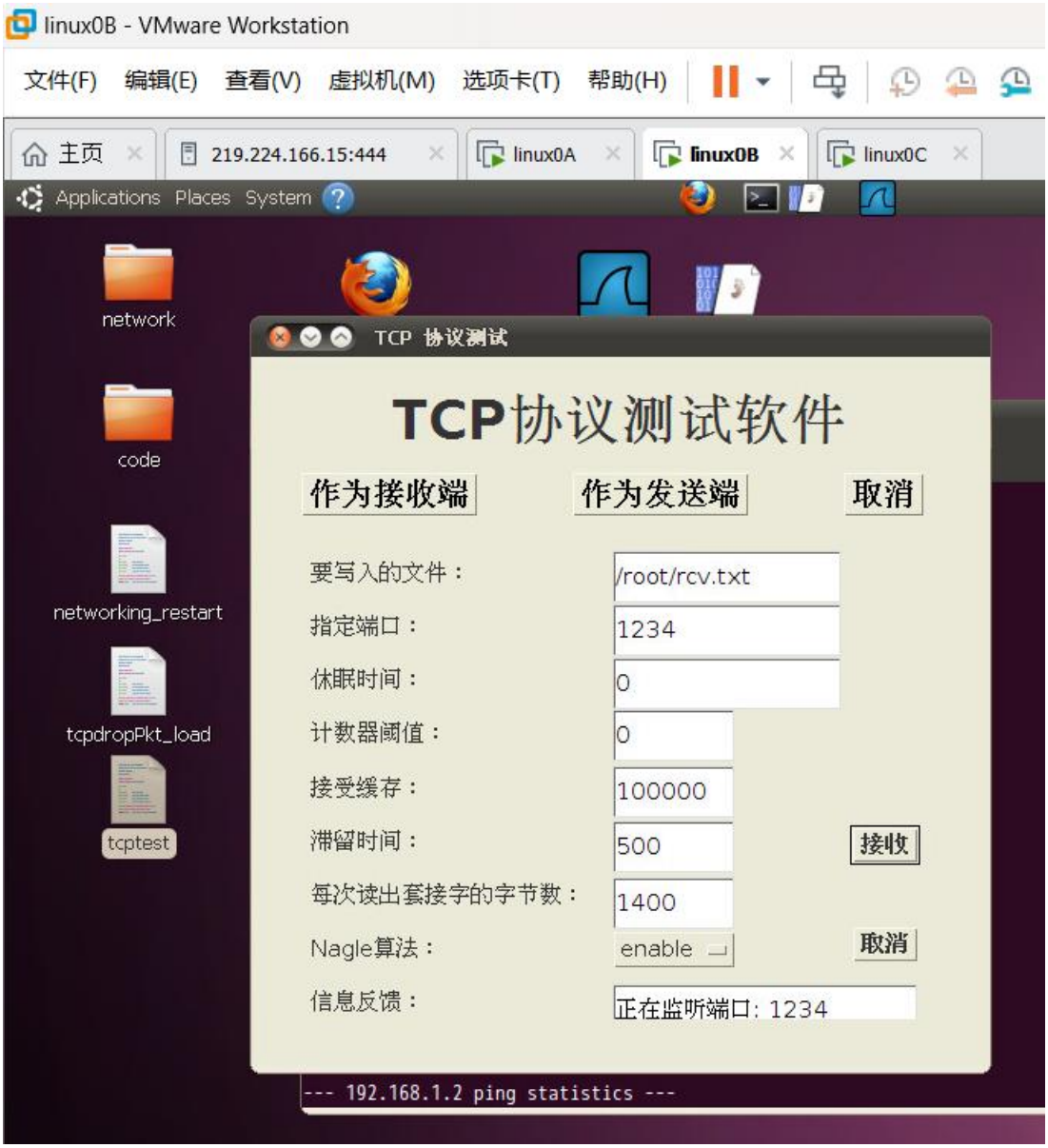
配置交换机、路由器


linux0A - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
主页
219.224.166.15:444
linux0A
linux0B
linux0C
Applications Places System
三 1月 3, 01:45 root
root@ubuntu: ~
File Edit View Terminal Help
* Copyright (c) 2004-2008 Hangzhou H3C Tech. Co., Ltd. All rights reserved. *
* Without the owner's prior written consent, *
* no decompiling or reverse-engineering shall be allowed. *
User interface aux0 is available.
Please press ENTER.
<s1>
%Apr 26 12:15:03:854 2000 s1 SHELL/4/LOGIN: Console login from aux0
<s1>sys
System View: return to User View with Ctrl+Z.
[s1]
[s1]vlan 2
[s1-vlan2]port e1/0/1 to e1/0/2
[s1-vlan2]quit
[s1]vlan 3
[s1-vlan3]port e1/0/23 to e1/0/24
[s1-vlan3]quit
[s1]
连线组网
实验组网 一键还原
PCA PCB PCC PCD

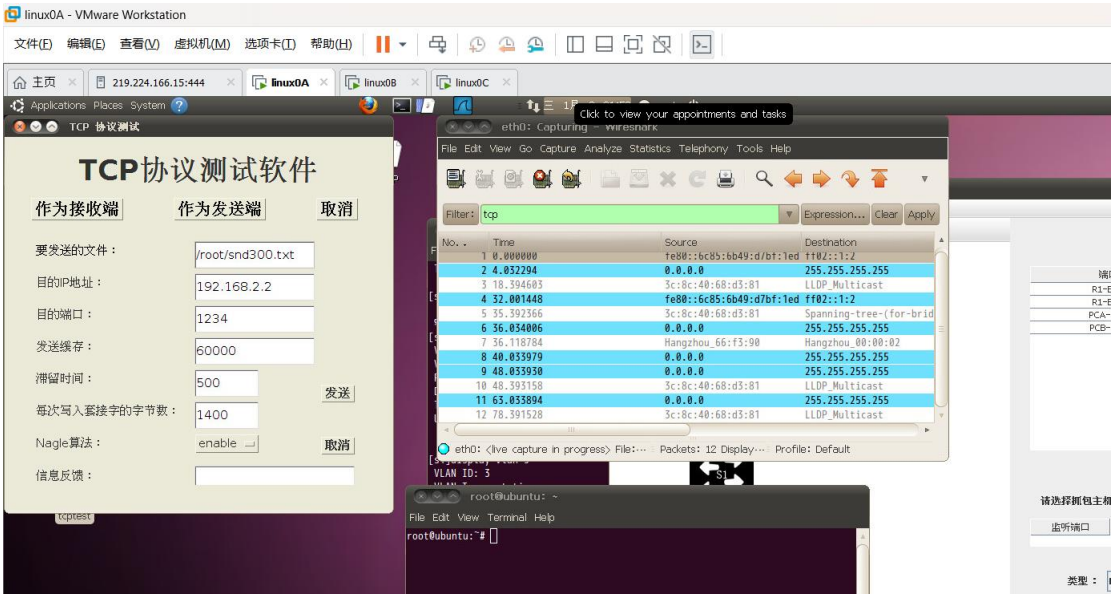

linux0C - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
主页
219.224.166.15:444
linux0A
linux0B
linux0C
Applications Places System
network
code
networking_re
tcptest
root@ubuntu: ~
File Edit View Terminal Help
System View: return to User View with Ctrl+Z.
[H3C]int e0/0
^
% Wrong parameter found at '^' position.
[H3C]inter e0/0
^
% Wrong parameter found at '^' position.
[H3C]interface e0/0
^
% Wrong parameter found at '^' position.
[H3C]interface ethernet0/0
^
% Wrong parameter found at '^' position.
[H3C]interface g0/0
[H3C-GigabitEthernet0/0]ip add 192.168.1.1 24
[H3C-GigabitEthernet0/0]quit
[H3C]int g0/1
[H3C-GigabitEthernet0/1]ip add 192.168.2.1 24
[H3C-GigabitEthernet0/1]
%Jan 3 10:13:50:271 2024 H3C IFNET/4/UPDOWN:
Line protocol on the interface GigabitEthernet0/1 is UP
[H3C-GigabitEthernet0/1]
[H3C-GigabitEthernet0/1]quit
[H3C]


linux0B - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
主页
219.224.166.15:444
linux0A
linux0B
linux0C
Applications Places System
Terminal
Use the command line
network
Firefox Web Browser
Wireshark
gFTP
code
networking_restart
tcpdropPkt_load
tcptest
root@ubuntu: ~
File Edit View Terminal Help
root@ubuntu:~#
root@ubuntu:~#
root@ubuntu:~# ping 192.168.1.2
PING 192.168.1.2 (192.168.1.2) 56(84) bytes of data.
64 bytes from 192.168.1.2: icmp_seq=1 ttl=63 time=3.64 ms
64 bytes from 192.168.1.2: icmp_seq=2 ttl=63 time=0.993 ms
64 bytes from 192.168.1.2: icmp_seq=3 ttl=63 time=0.896 ms
64 bytes from 192.168.1.2: icmp_seq=4 ttl=63 time=0.909 ms
64 bytes from 192.168.1.2: icmp_seq=5 ttl=63 time=0.921 ms
64 bytes from 192.168.1.2: icmp_seq=6 ttl=63 time=1.82 ms
64 bytes from 192.168.1.2: icmp_seq=7 ttl=63 time=0.907 ms
64 bytes from 192.168.1.2: icmp_seq=8 ttl=63 time=0.911 ms
64 bytes from 192.168.1.2: icmp_seq=9 ttl=63 time=0.922 ms
64 bytes from 192.168.1.2: icmp_seq=10 ttl=63 time=0.903 ms
64 bytes from 192.168.1.2: icmp_seq=11 ttl=63 time=0.888 ms
64 bytes from 192.168.1.2: icmp_seq=12 ttl=63 time=1.13 ms

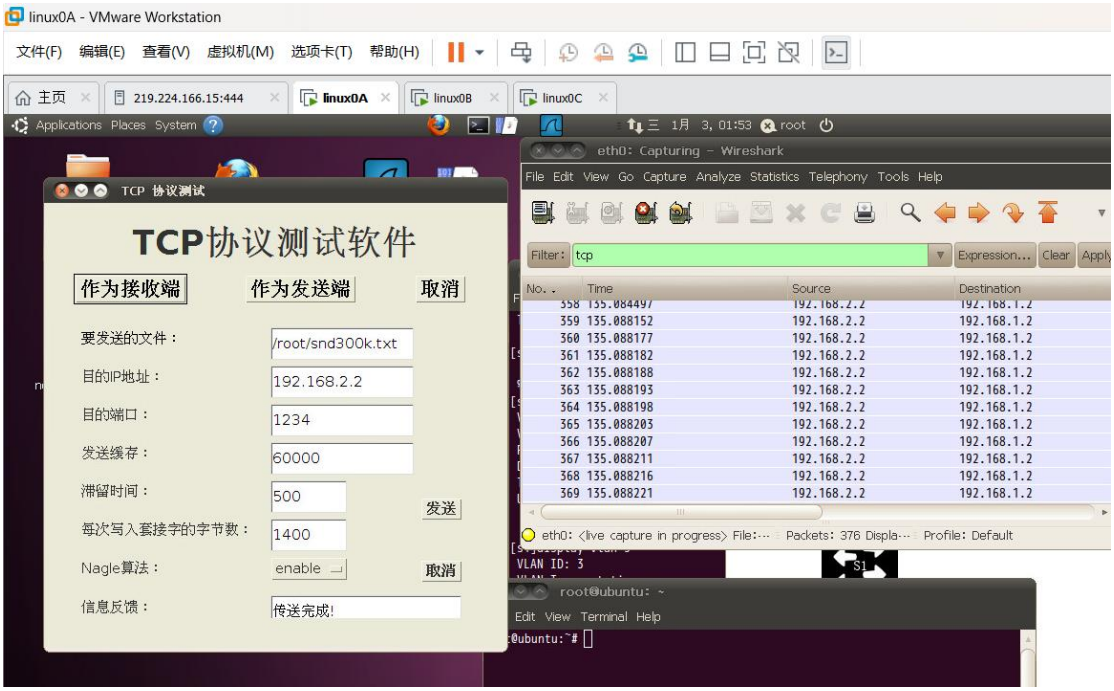
配置接收端


linux0B - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
主页
219.224.166.15:444
linux0A
linux0B
linux0C
Applications Places System
network
code
networking_restart
tcpdropPkt_load
tcptest
TCP 协议测试
TCP协议测试软件
作为接收端
作为发送端
取消
要写入的文件：
/root/rcv.txt
指定端口：
1234
休眠时间：
0
计数器阈值：
0
接受缓存：
100000
滞留时间：
500
接收
每次读出套接字的字节数：
1400
Nagle算法：
enable
取消
信息反馈：
正在监听端口: 1234
--- 192.168.1.2 ping statistics ---

配置发送端


linux0A - VMware Workstation
TCP协议测试软件
作为接收端
作为发送端
取消
要发送的文件：
/root/snd300.txt
目的IP地址：
192.168.2.2
目的端口：
1234
发送缓存：
60000
滞留时间：
500
发送
每次写入套接字的字节数：
1400
Nagle算法：
enable
取消
信息反馈：

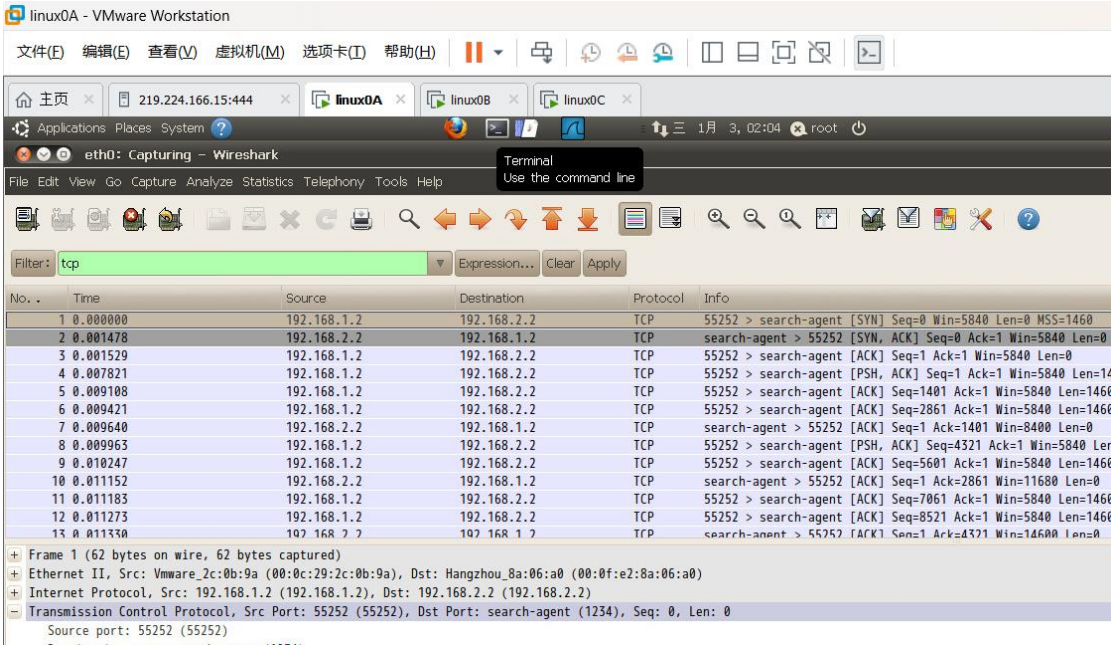
发送



2、根据 2.6 中步骤 5，结合预习报告，分析 TCP 连接的建立过程，根据 TCP 建立过程的三个报文，先填写下表：

字段名称	第一条报文	第二条报文	第三条报文
报文序号	1	2	3
Sequence Number	0	0	1
Acknowledgement Number	0	1	1
Ack	0	1	1
Syn	1	1	0

三次握手如下



3、根据 2.6 中步骤 6 回答：

TCP 连接建立时，其报文首部与其它 TCP 报文不同，有一个“Option”字段，它的作用是什么，值为多少？结合 IEEE802.3 协议规定的以太网最大帧长度分析此数据是怎样得出的。

答：在“Option”字段中，有一个参数为最大报文段长度（MSS），这一参数定义了可以传输的最大数据段的大小。在进行通信时，会比较客户端和服务器端各自支持的 MSS，并选择两者之中的较小值以便使用。在数据传输过程中，MSS 扮演着重要的角色。在本次实验中，记录的 MSS 值为 1460 字节。

计算 MSS 的方法是从最大传输单元（MTU）的长度中减去 IP 头部的标准长度和 TCP 头部的标准长度。具体的计算公式是：MSS = 最大 MTU - IP 头部长度 - TCP 头部长度。以常见的 MTU 值 1500 字节为例，减去 IP 和 TCP 头部各自的 20 字节，因此 MSS 计算结果为 1460 字节，即 MSS = 1500 - 20 - 20 = 1460。

4、根据 2.6 中步骤 7：结合预习报告，分析 TCP 连接的释放过程，选择 TCP 连接撤消的四个报文，将报文信息填入下表。

字段名称	第一条报文	第二条报文	第三条报文	第四条报文
报文序号	360	368	369	370
Sequence Number	354981	1	1	355912
Acknowledgement Number	1	355912	355912	2
Ack	1	1	1	1
Fin	1	0	1	0

分析如下：

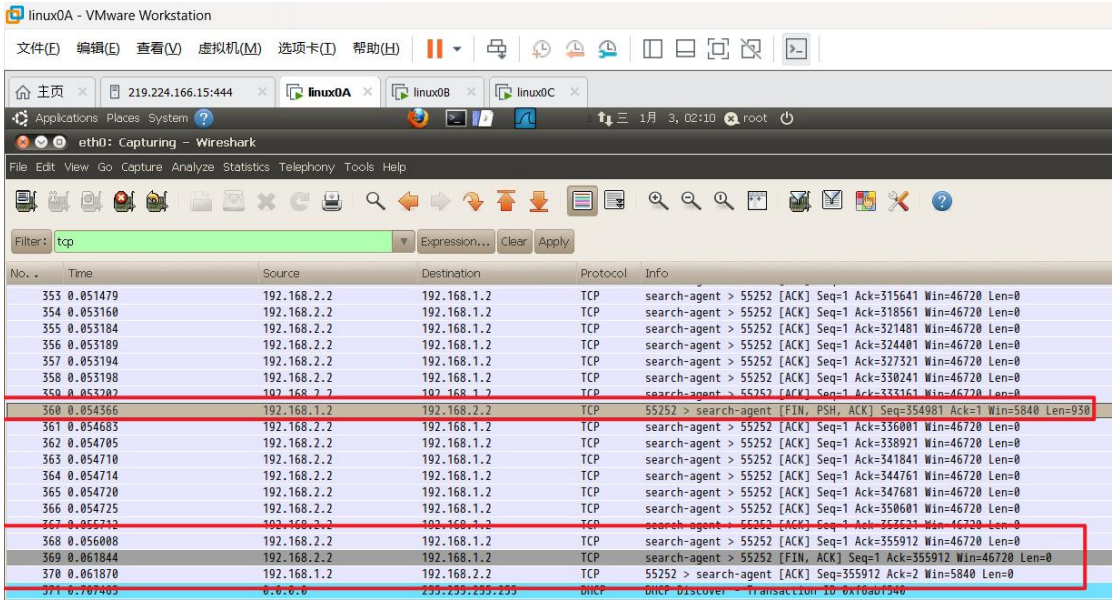
1. **第一条报文**：这是由客户端发起的连接释放请求，客户端发送了一个 FIN 标志位为 1 的报文。这表明客户端已经没有数据要发送，希望开始关闭连接。报文序号为 360，Sequence Number 为 354981，Acknowledgement Number 为 1（这里的 1 可能表示已经收到对方所有数据的确认，Ack 字段为 1 表示这是一个确认报文）。

2. **第二条报文**：作为响应，服务器认收到了客户端的 FIN 报文。服务器发送了一个 Ack 标志位为 1 的报文，其 Acknowledgement Number 为 355912，表示服务器确认收到了客户端发起的序号为 354981 的 FIN 报文。Sequence Number 为 1，这里的序号可能是服务器端的初始序号。此时，Fin 标志位为 0，表示服务器还没有准备好发送自己的 FIN 报文。

3. **第三条报文**：服务器准备关闭连接，并发送了自己的 FIN 报文。这个报文的 Sequence Number 为 1，Acknowledgement Number 为 355912，Ack 标志位为 1，表示确认之前收到的客户端的报文，而 Fin 标志位为 1，表示服务器也想结束连接。

4. **第四条报文**：最后，客户端确认收到服务器的 FIN 报文。客户端发送了一个 Acknowledgement Number 为 2 的确认报文，其 Sequence Number 为 355912，Ack 标志位为 1。这表明客户端已经收到了服务器希望关闭连接的信号，并且同意了连接的释放。

四次挥手如下：



5、根据 2.6 中步骤 8：分析 TCP 数据传送阶段的前 8 个报文，将报文信息填入下表。

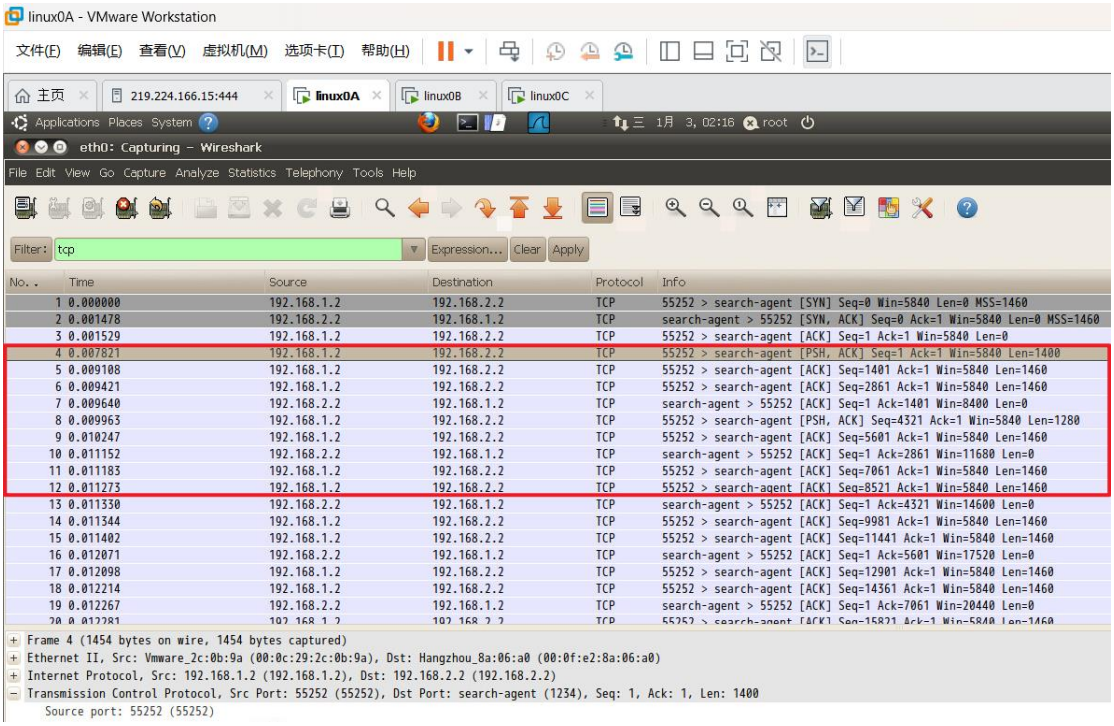
报文序号	报文种类(发送/确认)	序号字段	确认号字段	数据长度	被确认报文序号	窗口
4	发送和确认	1	1	1400	2	5840
5	发送和确认	1401	1	1460	2	5840
6	发送和确认	2861	1	1460	2	5840
7	确认	1	1401	0	4	8400
8	发送和确认	4321	1	1280	2	5840
9	发送和确认	5601	1	1460	2	5840
10	确认	1	2861	0	5	11680
11	发送和确认	7061	1	1460	2	5840

请写出 TCP 数据部分长度的计算公式。数据传送阶段第一个报文的序号字段值是否等于连接建立时第三个报文的序号？

答：TCP 数据部分长度 ＝IP 总长度 - IP 首都长度 - TCP 首部长度

是的，数据传送阶段第一个报文的序号字段值等于连接建立时第三个报文的序号，在实验中均为 1。

前 8 个报文如下：

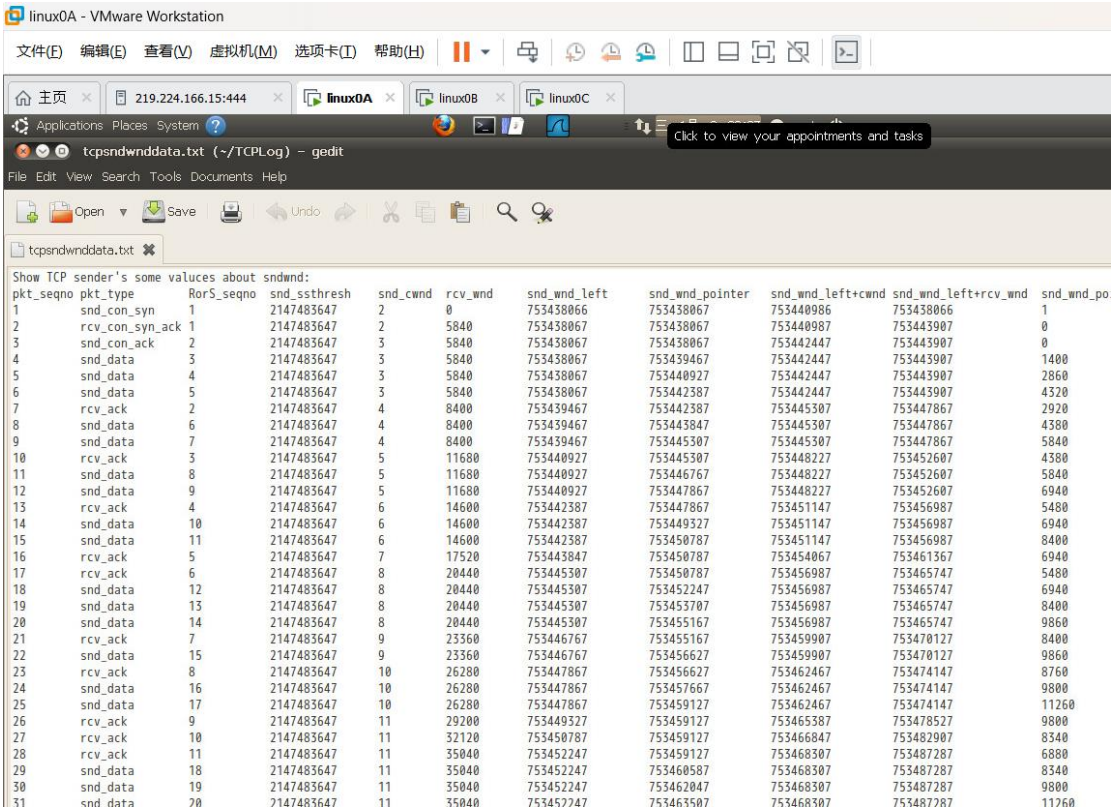


6. 根据 3.6.1 中“ 滑动窗口机制和窗口侦查机制分析”步骤 6 回答：

（1） 分析数据发送部分的前几条报文，描述发送方发送窗口的变化，并解释为什么？

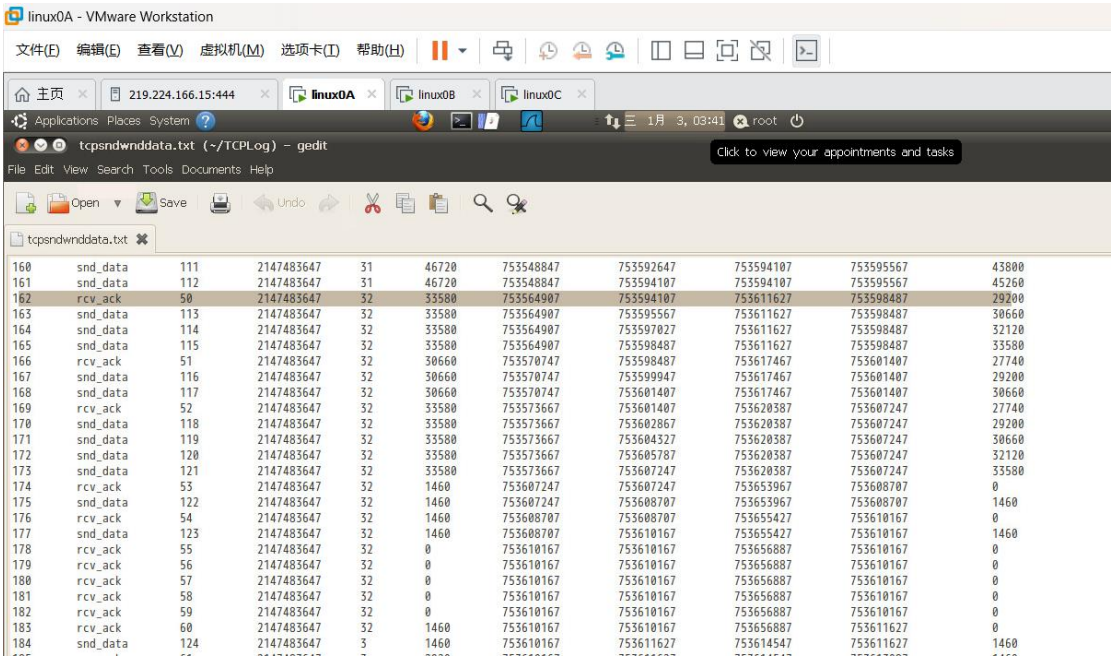
随着慢启动阶段的进行，由于拥塞窗口小于接收方窗口，因此发送方窗口会逐渐增大。这是因为发送方窗口的大小取决于拥塞窗口和接收窗口的最小值，随着收到确认报文，拥塞窗口逐渐增大，进而导致发送方窗口的增大。

如图所示，发送方拥塞窗口一直在增大。

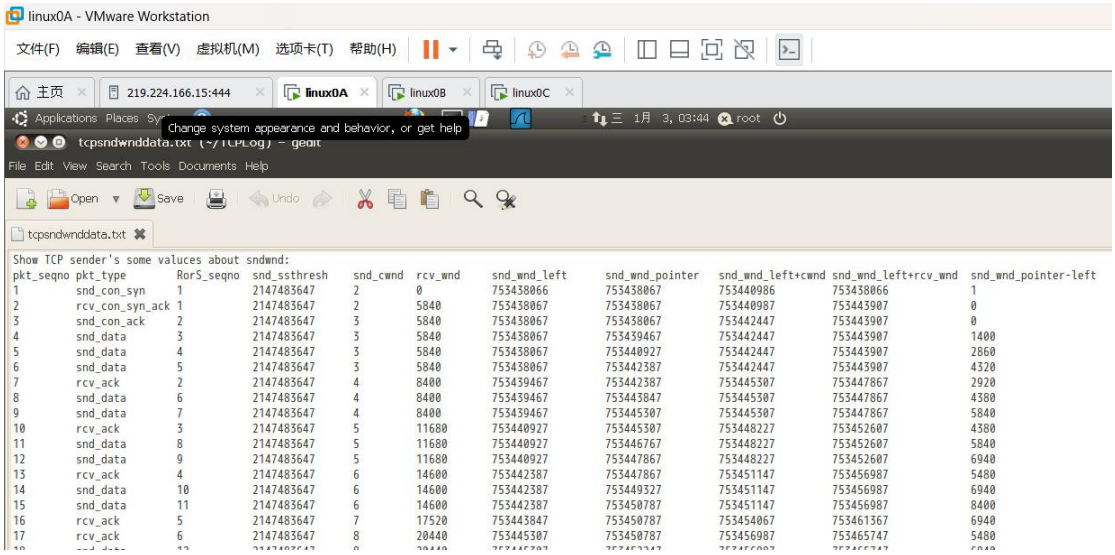


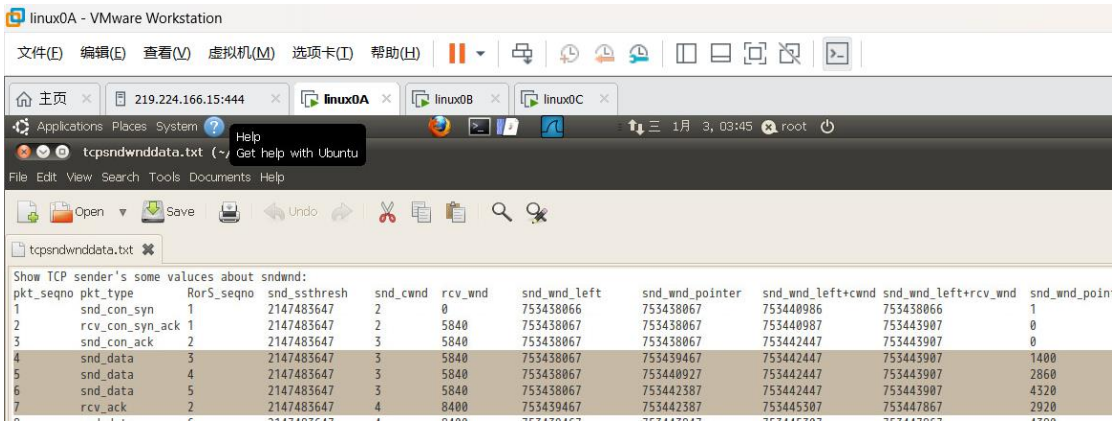
（2） 指出从哪个序号的报文能够看出接收端开始休眠，并解释理由。

162，如图所示，从 162 开始接受窗口不断减小，说明开始休眠.



（3） 分析文件 send2-组座号-tcpsndwnddata.txt，选中三次握手连接建立后的前 4 条报文记录（3 条 DATA 报文、1 条 ACK 报文，序号为 4、5、6、7），记下发送方发送窗口的相关值（rcv_wnd，snd_wnd_left，snd_wnd_point，snd_wnd_left+cwnd，snd_wnd_left+rcv_wnd , (snd_wnd_point- left)）。按下表分析计算接收方（及发送方）的窗口的相关值。





5 号报文（sender----data---->receiver）

	rcv_wnd	snd_wnd_left	snd_wnd_pointer	snd_wnd_left+cwnd 和 snd_wnd_left+rcv_wnd	snd_wnd_point- left
发送方发出报文	5840	753438067	753440927	753442447，753443907	2860
发送窗口右边沿	753442447				
	通告的接收窗口	接收窗口左边沿	接收窗口指针	接收窗口右边沿	在接收缓存中的数据量（即未确认的数据）
接收方接到 DATA 前	5840	753438067	753439467	753443907	2860

接收方接到DATA后	5840	753438067	753440927	753443907	2860

6 号报文（sender----data---->receiver）

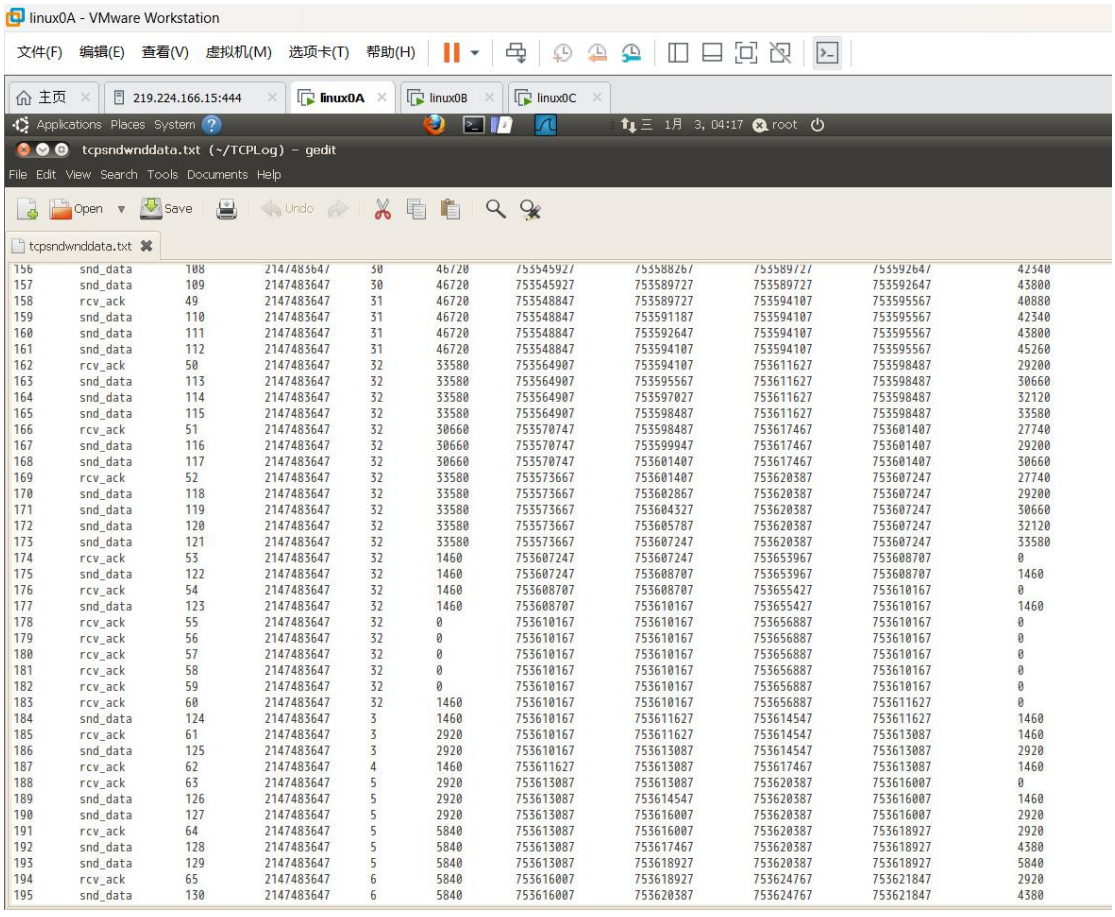
	rcv_wnd	snd_wnd_left	snd_wnd_pointer	snd_wnd_left+cwnd 和 snd_wnd_left+rcv_wnd	snd_wnd_point- left
发送方发出报文	5840	753438067	753442387	753442447，753443907	4320
发送窗口右边沿	753442447				
	通告的接收窗口	接收窗口左边沿	接收窗口指针	接收窗口右边沿	在接收缓存中的数据量（即未确认的数据）
接收方接到DATA前	5840	753438067	753440927	753443907	2860
接收方接到DATA后	5840	753438067	753442387	753443907	4320

7 号报文（receiver ----ack----> sender）

	通告的接收窗口	接收窗口左边沿	接收窗口指针	接收窗口右边沿	在接收缓存中的数据量（即未确认的数据）
接收方发出 ACK	8400	753439467	753442387	753447876	2920
	rcv_wnd	snd_wnd_left	snd_wnd_pointer	snd_wnd_left+cwnd 和 snd_wnd_left+rcv_wnd	snd_wnd_point- left
发送方接到 ACK 后	8400	753439467	7533443847	753445307，753447876	4380
发送窗口右边沿	753447876				
发送方接到 ACK 前	5840	753438067	753442387	753442447，753443907	4320
发送窗口右边沿	753443907				

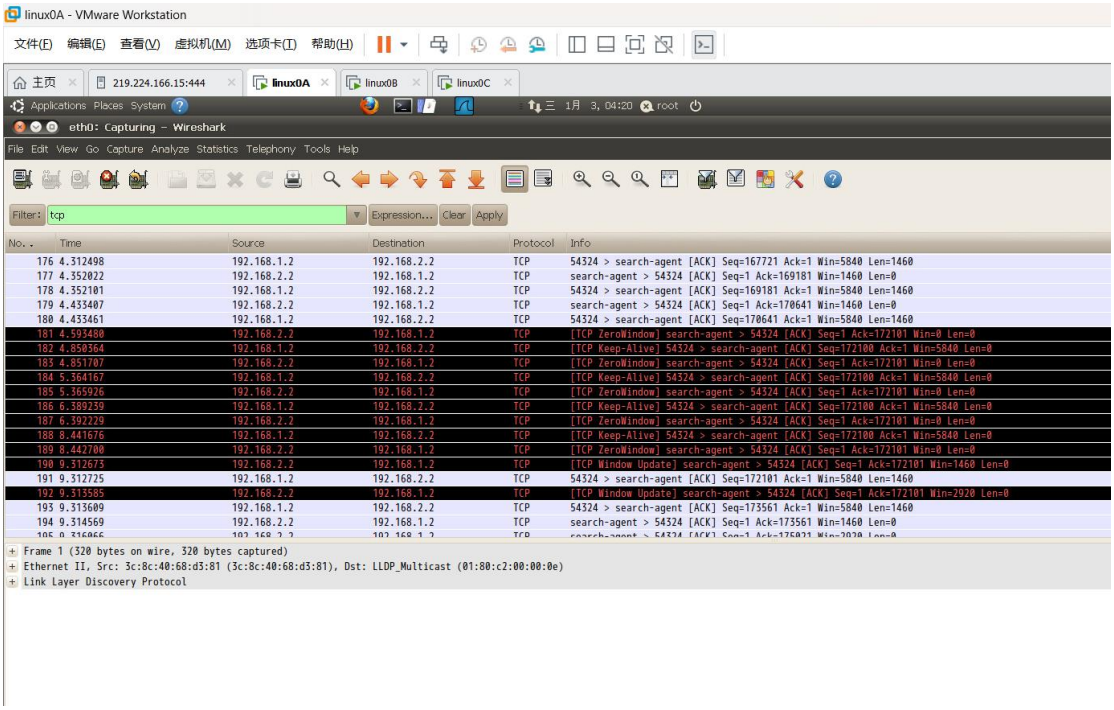
（4） 根据文件 send2-组座号-tcpsndwnddata.txt 中发送方的发送窗口相关值进行分析，接收方开始休眠后，描述接收窗口的变化，指出窗口收缩、窗口合拢、窗口张开对应的开始报文序号，并记下 send2-组座号-tcpsndwnddata.txt 文件中的对应报文的数值记录（pkt_seqno，pkt_type，..，……）。

如图所示：161 号报文后窗口开始合拢，182 号报文后窗口开始打开

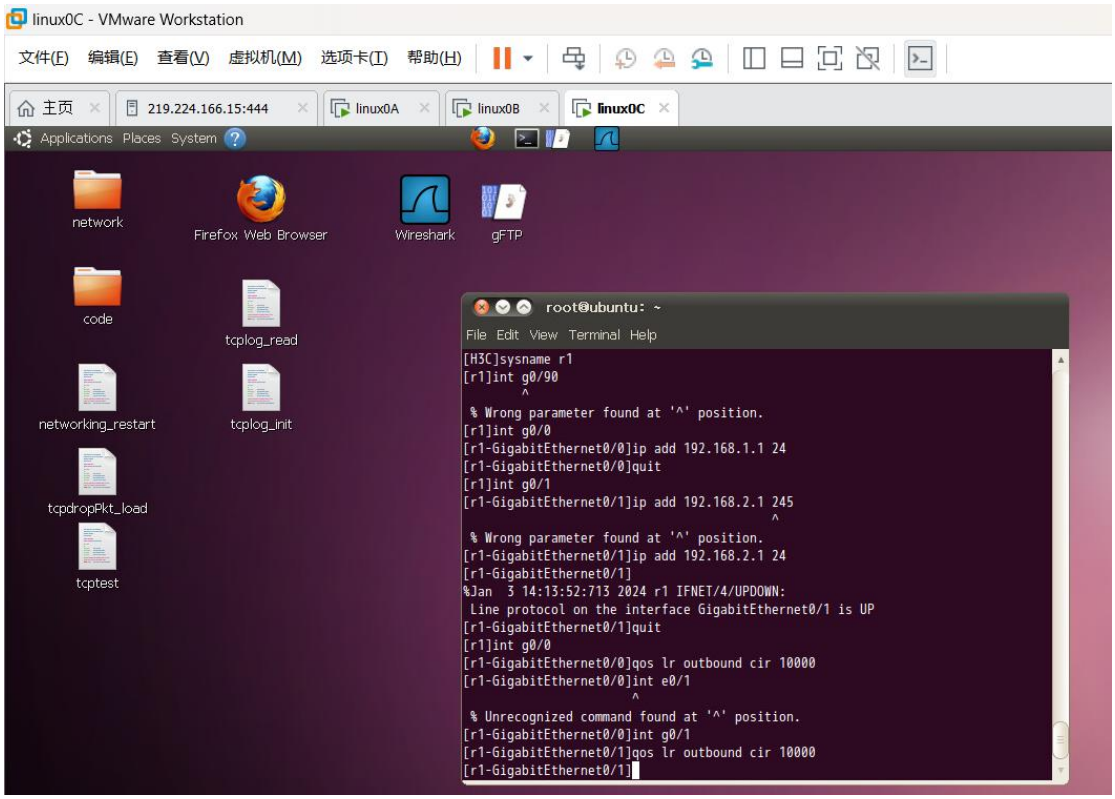


156	snd_data	108	2147483647	30	46720	753545927	753588267	753589727	753592647	42340
157	snd_data	109	2147483647	30	46720	753545927	753589727	753589727	753592647	43800
158	rcv_ack	49	2147483647	31	46720	753548847	753589727	753594107	753595567	40880
159	snd_data	110	2147483647	31	46720	753548847	753591187	753594107	753595567	42340
160	snd_data	111	2147483647	31	46720	753548847	753592647	753594107	753595567	43800
161	snd_data	112	2147483647	31	46720	753548847	753594107	753594107	753595567	45260
162	rcv_ack	50	2147483647	32	33580	753564907	753594107	753611627	753598487	29200
163	snd_data	113	2147483647	32	33580	753564907	753595567	753611627	753598487	30660
164	snd_data	114	2147483647	32	33580	753564907	753597027	753611627	753598487	32120
165	snd_data	115	2147483647	32	33580	753564907	753598487	753611627	753598487	33580
166	rcv_ack	51	2147483647	32	30660	753570747	753598487	753617467	753601407	27740
167	snd_data	116	2147483647	32	30660	753570747	753599947	753617467	753601407	29200
168	snd_data	117	2147483647	32	30660	753570747	753601407	753617467	753601407	30660
169	rcv_ack	52	2147483647	32	33580	753573667	753601407	753620387	753607247	27740
170	snd_data	118	2147483647	32	33580	753573667	753602867	753620387	753607247	29200
171	snd_data	119	2147483647	32	33580	753573667	753604327	753620387	753607247	30660
172	snd_data	120	2147483647	32	33580	753573667	753605787	753620387	753607247	32120
173	snd_data	121	2147483647	32	33580	753573667	753607247	753620387	753607247	33580
174	rcv_ack	53	2147483647	32	1460	753607247	753607247	753653967	753608707	0
175	snd_data	122	2147483647	32	1460	753607247	753608707	753653967	753608707	1460
176	rcv_ack	54	2147483647	32	1460	753608707	753608707	753655427	753610167	0
177	snd_data	123	2147483647	32	1460	753608707	753610167	753655427	753610167	1460
178	rcv_ack	55	2147483647	32	0	753610167	753610167	753656887	753610167	0
179	rcv_ack	56	2147483647	32	0	753610167	753610167	753656887	753610167	0
180	rcv_ack	57	2147483647	32	0	753610167	753610167	753656887	753610167	0
181	rcv_ack	58	2147483647	32	0	753610167	753610167	753656887	753610167	0
182	rcv_ack	59	2147483647	32	0	753610167	753610167	753656887	753610167	0
183	rcv_ack	60	2147483647	32	1460	753610167	753610167	753656887	753611627	0
184	snd_data	124	2147483647	3	1460	753610167	753611627	753614547	753611627	1460
185	rcv_ack	61	2147483647	3	2920	753610167	753611627	753614547	753613087	1460
186	snd_data	125	2147483647	3	2920	753610167	753613087	753614547	753613087	2920
187	rcv_ack	62	2147483647	4	1460	753611627	753613087	753617467	753613087	1460
188	rcv_ack	63	2147483647	5	2920	753613087	753613087	753620387	753616007	0
189	snd_data	126	2147483647	5	2920	753613087	753614547	753620387	753616007	1460
190	snd_data	127	2147483647	5	2920	753613087	753616007	753620387	753616007	2920
191	rcv_ack	64	2147483647	5	5840	753613087	753616007	753620387	753618927	2920
192	snd_data	128	2147483647	5	5840	753613087	753617467	753620387	753618927	4380
193	snd_data	129	2147483647	5	5840	753613087	753618927	753620387	753618927	5840
194	rcv_ack	65	2147483647	6	5840	753616007	753618927	753624767	753621847	2920
195	snd_data	130	2147483647	6	5840	753616007	753620387	753624767	753621847	4380

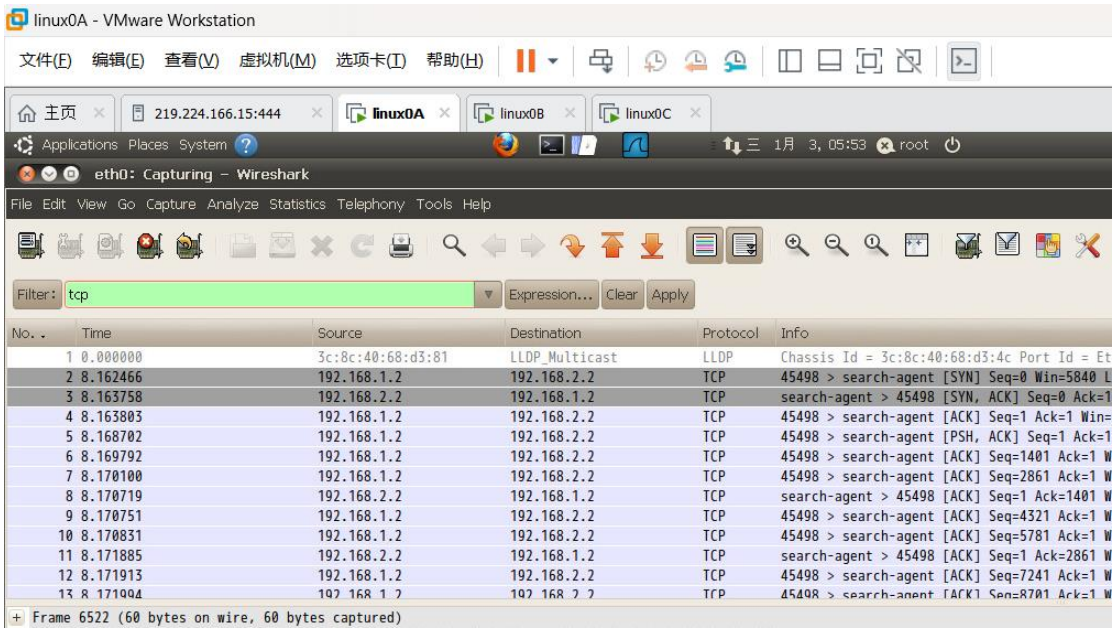
7. 根据 3.6.1 中“ 滑动窗口机制和窗口侦查机制分析”步骤 7 回答：
写出窗口侦查开始的报文序号，窗口侦查报文数据长度、窗口侦查报文发送的时间规律。
答：窗口侦查报文指的是 Keep-Alive 报文（黑底红字），
如图所示，窗口侦查开始的报文序号为 182
窗口侦查报文数据长度为 0
每相邻两条窗口侦查报文 Keep-Alive 报文时间差组成的数据序列的规律：第一个窗口探测报文如果没有收到窗口打开的确认，后续的探测报文发送间隔会逐渐翻倍，直到达到一个预设的最大值.

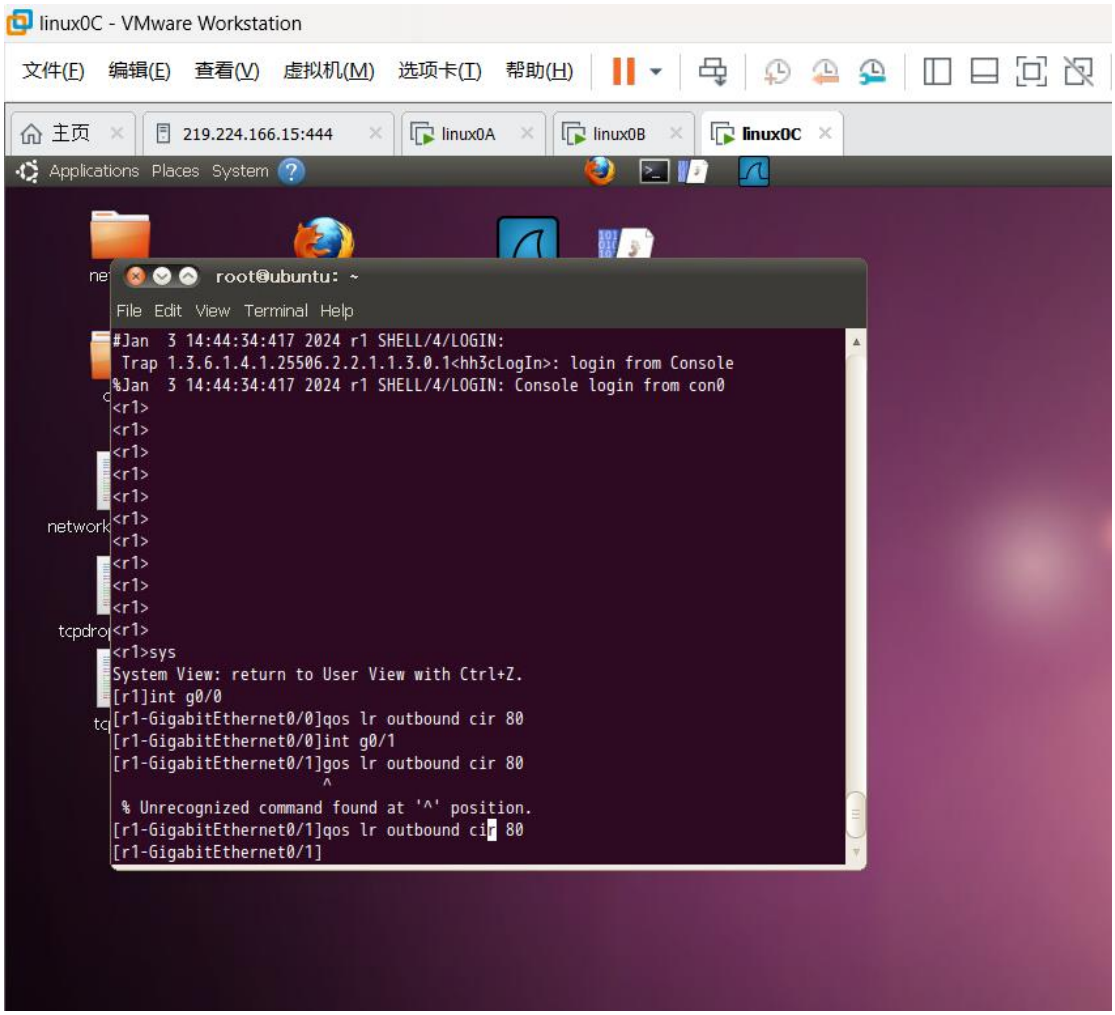


8. 根据 3.6.2 中“ 慢启动、拥塞避免及拥塞处理和超时与重传机制分析”步骤 9 回答：
设置转发速率


linux0C - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
主页
219.224.166.15:444
linux0A
linux0B
linux0C
Applications Places System
network
Firefox Web Browser
Wireshark
gFTP
code
tcplog_read
networking_restart
tcplog_init
tcpdropPkt_load
tcptest
root@ubuntu: ~
File Edit View Terminal Help
[H3C]sysname r1
[r1]int g0/90
^
% Wrong parameter found at '^' position.
[r1]int g0/0
[r1-GigabitEthernet0/0]ip add 192.168.1.1 24
[r1-GigabitEthernet0/0]quit
[r1]int g0/1
[r1-GigabitEthernet0/1]ip add 192.168.2.1 245
^
% Wrong parameter found at '^' position.
[r1-GigabitEthernet0/1]ip add 192.168.2.1 24
[r1-GigabitEthernet0/1]
%Jan 3 14:13:52:713 2024 r1 IFNET/4/UPDOWN:
Line protocol on the interface GigabitEthernet0/1 is UP
[r1-GigabitEthernet0/1]quit
[r1]int g0/0
[r1-GigabitEthernet0/0]qos lr outbound cir 10000
[r1-GigabitEthernet0/0]int e0/1
^
% Unrecognized command found at '^' position.
[r1-GigabitEthernet0/0]int g0/1
[r1-GigabitEthernet0/1]qos lr outbound cir 10000
[r1-GigabitEthernet0/1]

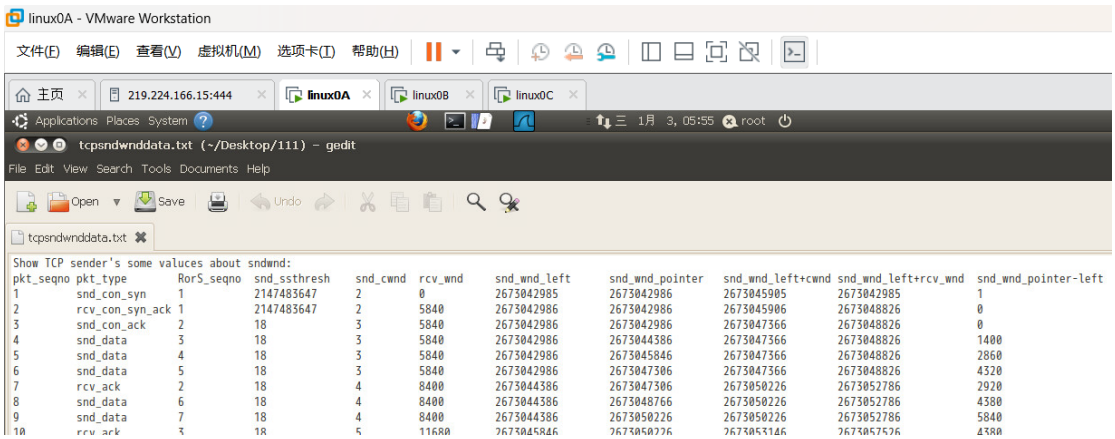

linux0A - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
主页
219.224.166.15:444
linux0A
linux0B
linux0C
Applications Places System
三 1月 3, 05:52 root
TCP 协议测试
TCP协议测试软件
作为接收端
作为发送端
取消
要发送的文件：
/root/snd6m.txt
目的IP地址：
192.168.2.2
目的端口：
1234
发送缓存：
60000
滞留时间：
500
发送
每次写入套接字的字节数：
1400
Nagle算法：
enable
取消
信息反馈：
传送完成!
Clear
Apply
Protocol
Info
TCP search-agent > 45498 [ACK
TCP 45498 > search-agent [FIN
TCP 45498 > search-agent [ACK
lticast (01:80:c2:00:00:0e)


linux0A - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
主页
219.224.166.15:444
linux0A
linux0B
linux0C
Applications Places System
eth0: Capturing - Wireshark
File Edit View Go Capture Analyze Statistics Telephony Tools Help
Filter: tcp
Expression... Clear Apply
No. . Time Source Destination Protocol Info
1 0.000000 3c:8c:40:68:d3:81 LLDP_Multicast LLDP Chassis Id = 3c:8c:40:68:d3:4c Port Id = Et
2 8.162466 192.168.1.2 192.168.2.2 TCP 45498 > search-agent [SYN] Seq=0 Win=5840 L
3 8.163758 192.168.2.2 192.168.1.2 TCP search-agent > 45498 [SYN, ACK] Seq=0 Ack=1
4 8.163803 192.168.1.2 192.168.2.2 TCP 45498 > search-agent [ACK] Seq=1 Ack=1 Win=
5 8.168702 192.168.1.2 192.168.2.2 TCP 45498 > search-agent [PSH, ACK] Seq=1 Ack=1
6 8.169792 192.168.1.2 192.168.2.2 TCP 45498 > search-agent [ACK] Seq=1401 Ack=1 W
7 8.170100 192.168.1.2 192.168.2.2 TCP 45498 > search-agent [ACK] Seq=2861 Ack=1 W
8 8.170719 192.168.2.2 192.168.1.2 TCP search-agent > 45498 [ACK] Seq=1 Ack=1401 W
9 8.170751 192.168.1.2 192.168.2.2 TCP 45498 > search-agent [ACK] Seq=4321 Ack=1 W
10 8.170831 192.168.1.2 192.168.2.2 TCP 45498 > search-agent [ACK] Seq=5781 Ack=1 W
11 8.171885 192.168.2.2 192.168.1.2 TCP search-agent > 45498 [ACK] Seq=1 Ack=2861 W
12 8.171913 192.168.1.2 192.168.2.2 TCP 45498 > search-agent [ACK] Seq=7241 Ack=1 W
Frame 6522 (60 bytes on wire, 60 bytes captured)


linux0C - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
主页
219.224.166.15:444
linux0A
linux0B
linux0C
Applications Places System
root@ubuntu: ~
File Edit View Terminal Help
#Jan 3 14:44:34:417 2024 r1 SHELL/4/LOGIN:
Trap 1.3.6.1.4.1.25506.2.2.1.1.3.0.1<hh3cLogIn>: login from Console
%Jan 3 14:44:34:417 2024 r1 SHELL/4/LOGIN: Console login from con0
<r1>
<r1>
<r1>
<r1>
<r1>
<r1>
<r1>
<r1>
<r1>
<r1>
<r1>
<r1>sys
System View: return to User View with Ctrl+Z.
[r1]int g0/0
[r1-GigabitEthernet0/0]qos lr outbound cir 80
[r1-GigabitEthernet0/0]int g0/1
[r1-GigabitEthernet0/1]gos lr outbound cir 80
^
% Unrecognized command found at '^' position.
[r1-GigabitEthernet0/1]qos lr outbound cir 80
[r1-GigabitEthernet0/1]



（1） 选中第一条发送数据的报文记录，记下其 ssthresh 和 cwnd 值是多少？为何为此值？按发送窗口的计算公式计算出当前的发送窗口 snd_wnd 值。并记下此时的发送窗口左边沿 snd_wnd_left 值并计算出此时的发送窗口右边沿。



ssthresh=2147483647

cwnd = 3

原因：

ssthresh 实际上等于 2^31 - 1，是一个整型变量可以表示的最大正值。在 TCP 拥塞控制的上下文中，这个值被设置为一个非常大的数，以便在开始 TCP 连接时，不会因为慢启动阈值的限制而影响拥塞窗口的增长。

cwnd = 3: 根据 TCP 的标准，拥塞窗口在连接开始时通常被初始化为一个较小的值，是为了慢慢探测网络的拥塞程度，避免一开始就发送太多数据而可能引起的网络拥塞。

snd_wnd=5840

snd_wnd_left=2673042986

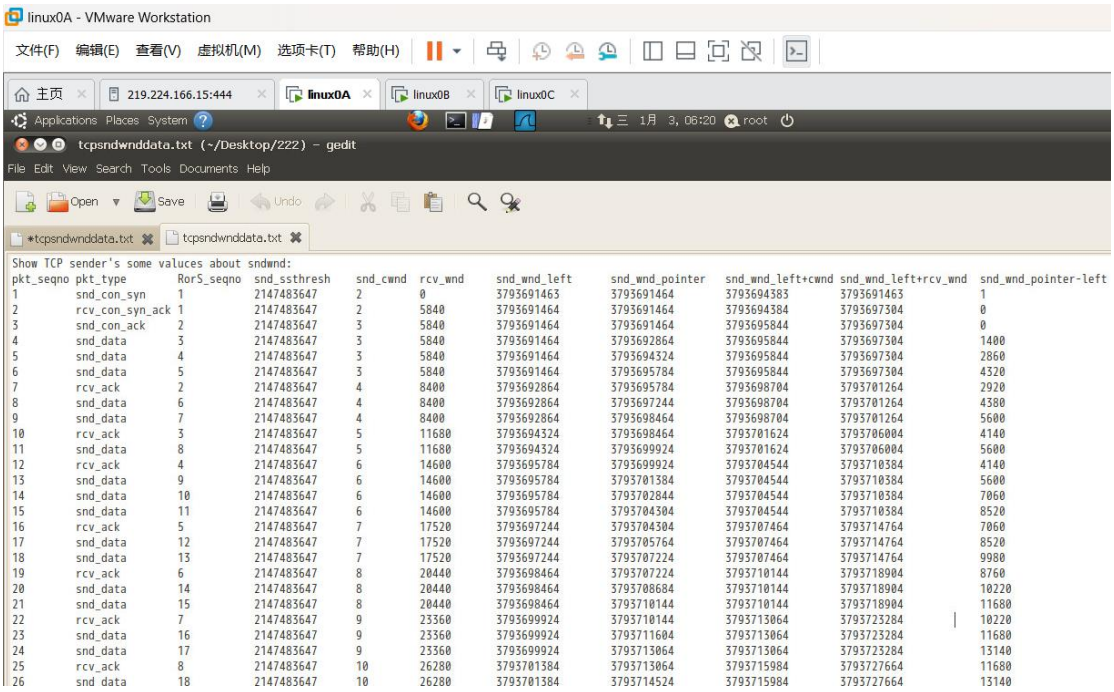
（2） 在随后的发送数据报文中，ssthresh 和 cwnd 值有何变化，呈何种规律？为什么呈此种规律？发送的报文是否可以验证这一规律？

Ssthresh 保持不变。

cwnd 不断增大

TCP 在连接过程的三次握手完成后，开始传数据，并不是一开始向网络通道中发送大量的数据包，这样很容易导致网络中路由器缓存空间耗尽，从而发生拥塞；而是根据初始的 cwnd 大小逐步增加发送的数据量，cwnd 初始化为 1 个最大报文段(MSS)大小；每当有一个报文段被确认，cwnd 大小指数增长。

可以，如图所示；

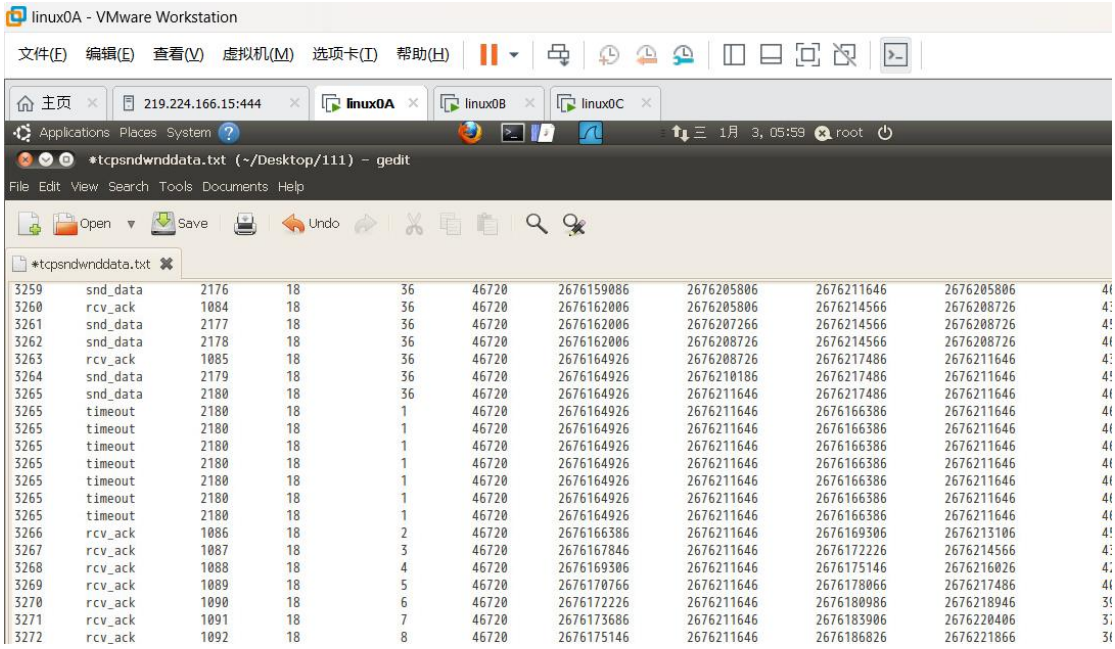


Show TCP sender's some valuces about sndwnd:

pkt_seqno	pkt_type	RorS_seqno	snd_ssthresh	snd_cwnd	rcv_wnd	snd_wnd_left	snd_wnd_pointer	snd_wnd_left+cwnd	snd_wnd_left+rcv_wnd	snd_wnd_pointer-left
1	snd_con_syn	1	2147483647	2	0	3793691463	3793691464	3793694385	3793691463	1
2	rcv_con_syn_ack	1	2147483647	2	5840	3793691464	3793691464	3793694384	3793697304	0
3	snd_con_ack	2	2147483647	3	5840	3793691464	3793691464	3793695844	3793697304	0
4	snd_data	3	2147483647	3	5840	3793691464	3793692864	3793695844	3793697304	1400
5	snd_data	4	2147483647	3	5840	3793691464	3793694324	3793695844	3793697304	2860
6	snd_data	5	2147483647	3	5840	3793691464	3793695784	3793695844	3793697304	4320
7	rcv_ack	2	2147483647	4	8400	3793692864	3793695784	3793698704	3793701264	2920
8	snd_data	6	2147483647	4	8400	3793692864	3793697244	3793698704	3793701264	4380
9	snd_data	7	2147483647	4	8400	3793692864	3793698464	3793698704	3793701264	5600
10	rcv_ack	3	2147483647	5	11680	3793694324	3793698464	3793701624	3793706004	4140
11	snd_data	8	2147483647	5	11680	3793694324	3793699924	3793701624	3793706004	5600
12	rcv_ack	4	2147483647	6	14600	3793695784	3793699924	3793704544	3793710384	4140
13	snd_data	9	2147483647	6	14600	3793695784	3793701384	3793704544	3793710384	5600
14	snd_data	10	2147483647	6	14600	3793695784	3793702844	3793704544	3793710384	7060
15	snd_data	11	2147483647	6	14600	3793695784	3793704304	3793704544	3793710384	8520
16	rcv_ack	5	2147483647	7	17520	3793697244	3793704304	3793707464	3793714764	7060
17	snd_data	12	2147483647	7	17520	3793697244	3793705764	3793707464	3793714764	8520
18	snd_data	13	2147483647	7	17520	3793697244	3793707224	3793707464	3793714764	9980
19	rcv_ack	6	2147483647	8	20440	3793698464	3793707224	3793710144	3793718904	8760
20	snd_data	14	2147483647	8	20440	3793698464	3793708684	3793710144	3793718904	10220
21	snd_data	15	2147483647	8	20440	3793698464	3793710144	3793710144	3793718904	11680
22	rcv_ack	7	2147483647	9	23360	3793699924	3793710144	3793713064	3793723284	10220
23	snd_data	16	2147483647	9	23360	3793699924	3793711604	3793713064	3793723284	11680
24	snd_data	17	2147483647	9	23360	3793699924	3793713064	3793713064	3793723284	13140
25	rcv_ack	8	2147483647	10	26280	3793701384	3793713064	3793715984	3793727664	11680
26	snd_data	18	2147483647	10	26280	3793701384	3793714524	3793715984	3793727664	13140

（3） 指出 ssthresh 和 cwnd 值有突然变化的报文序号，为何会有这种变化？此后（直至结束）的 ssthresh 和 cwnd 值有何变化，呈何种规律？为什么？

如图所示，cwnd 突然变小，是因为发生了拥塞（手动断开开关），随后继续慢慢增长。



9. 根据 3.6.2 中“ 慢启动、拥塞避免及拥塞处理和超时与重传机制分析”步骤 10 回答：

（1） 对 Wireshark 截获的报文进行分析，分别记下 TCP 传输过程中发送第一条重传报文的序号，并请用截获报文的时间字段分别计算出在两种转发速率下第一条重传报文和其对应的原始发送报文的时间差、第二条重传报文和其对应的原始发送报文的时间差。分析同种速率下和两种转发速率下两次重传的时间差，结合题（2）对此现象做出解释。

第一种情况：

原始报文：报文序号=3197，seq=3121941，时刻=10.202247

第一次重传：报文序号=3200，seq=3121941，时刻=10.429111

第二次重传：报文序号=3201，seq=3121941，时刻=10.881627

第三次重传：报文序号=3201，seq=3121941，时刻=11.785393

第二种情况：

原始报文：报文序号=23，seq=20141，时刻=0.245210

第一次重传：报文序号=52，seq=20141，时刻=4.682593

第二次重传：报文序号=53，seq=20141，时刻=10.957454

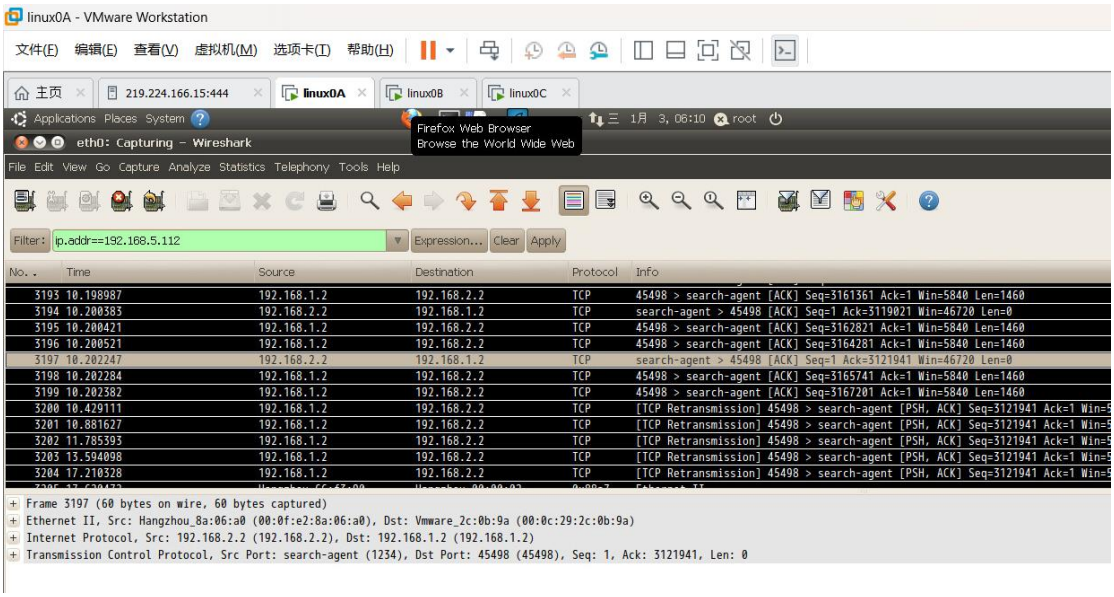
第三次重传：报文序号=61，seq=20141，时刻=23.507405

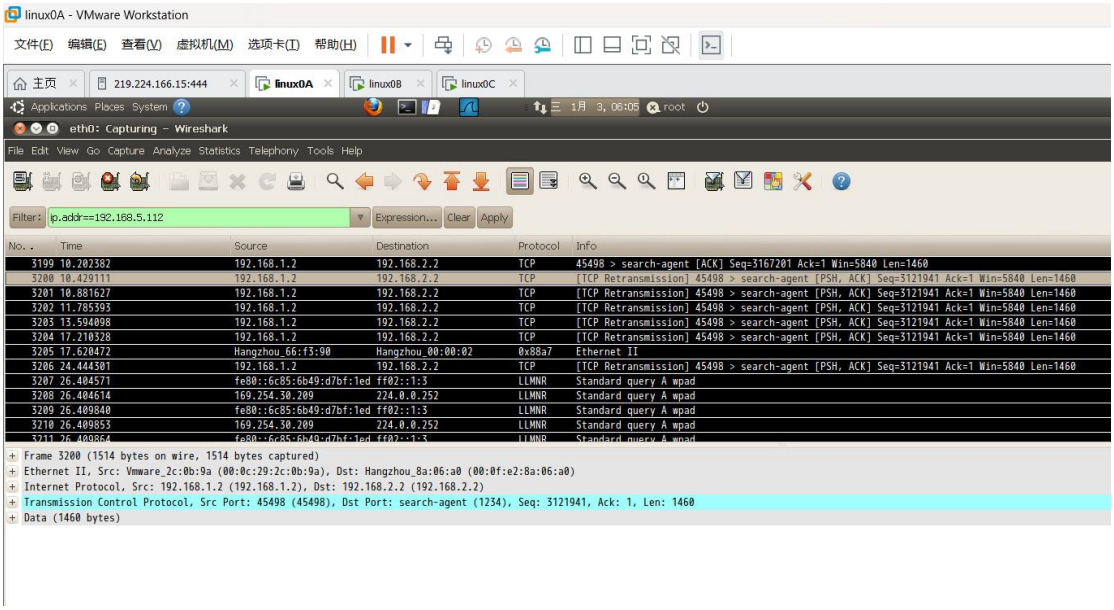
	第一个重传时间差	第二个重传时间差	第三个重传时间差
10Mbps	0.22686 秒	0.45252 秒	0.90377 秒
80Kbps	4.43738 秒	6.27486 秒	12.55 秒

解释：

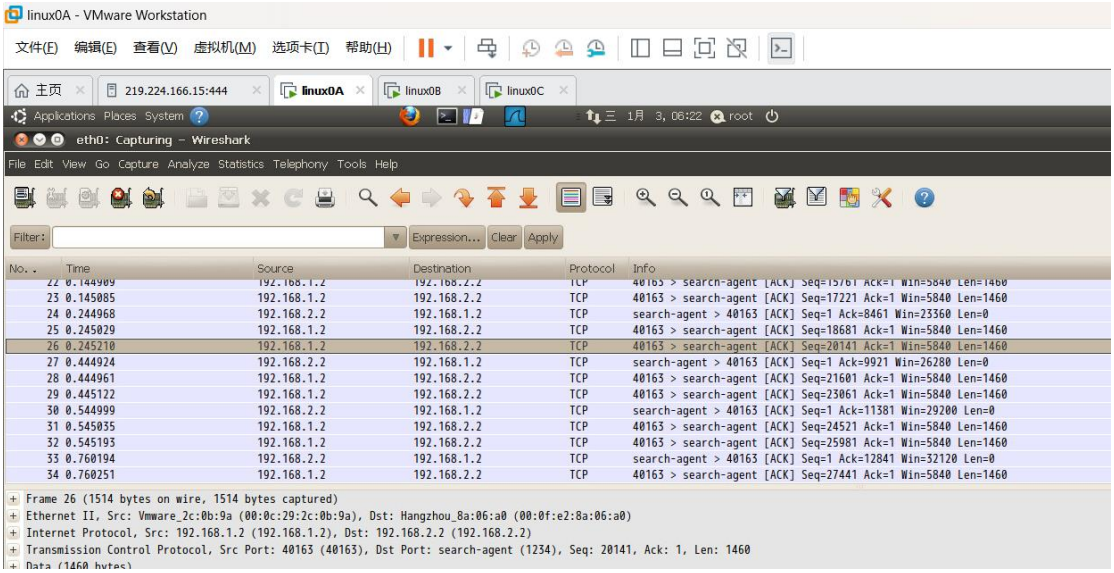
可以发现，时间差几乎是翻倍增长，经查阅，在 TCP 协议中，当网络出现拥塞时，TCP 会采用一种叫做"指数退避"的策略来处理。这个策略的基本思想就是，每次数据包传输失败后，TCP 会将下一次重传的时间间隔加倍。这样做的目的是为了给网络足够的时间来恢复，避免因为过多的数据包同时发送而导致的更严重的网络拥塞。

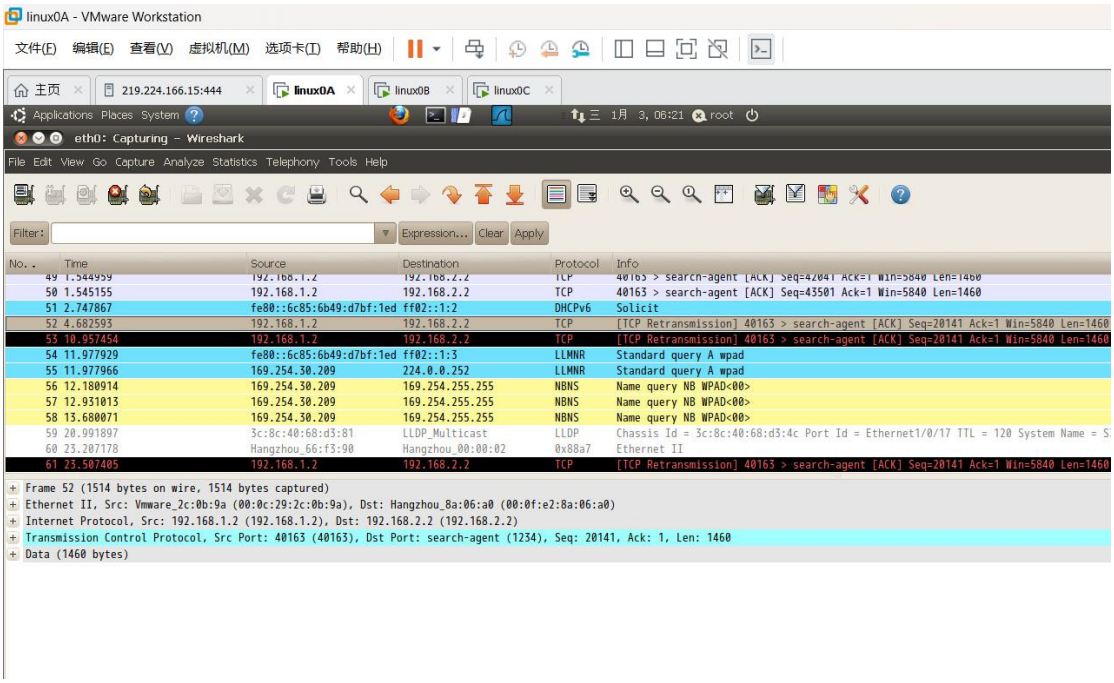
10mbps 的情况：





80kbps 的情况：





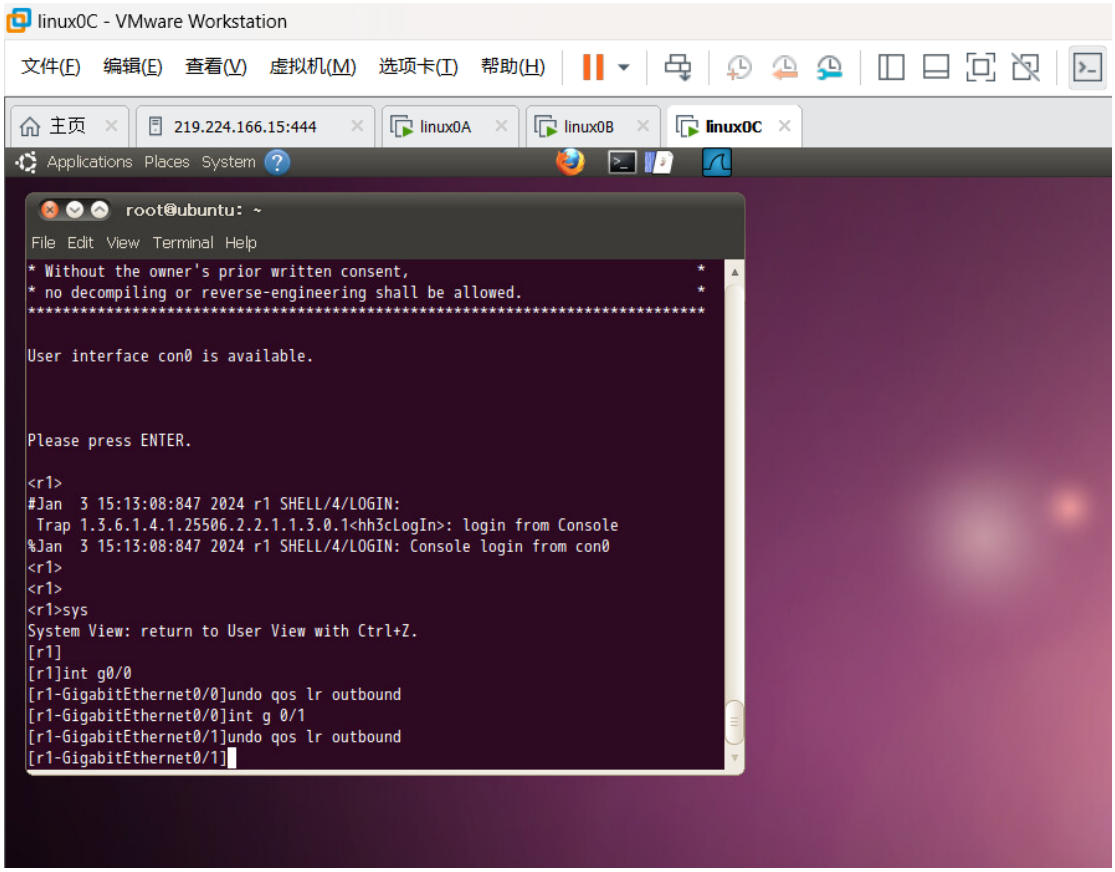
10. 根据 3.6.3 中步骤 1，继续上一节的实验，配置路由器，取消对接口的速率限制。请写出命令：

[Router]interface GigabitEthernet 0/0

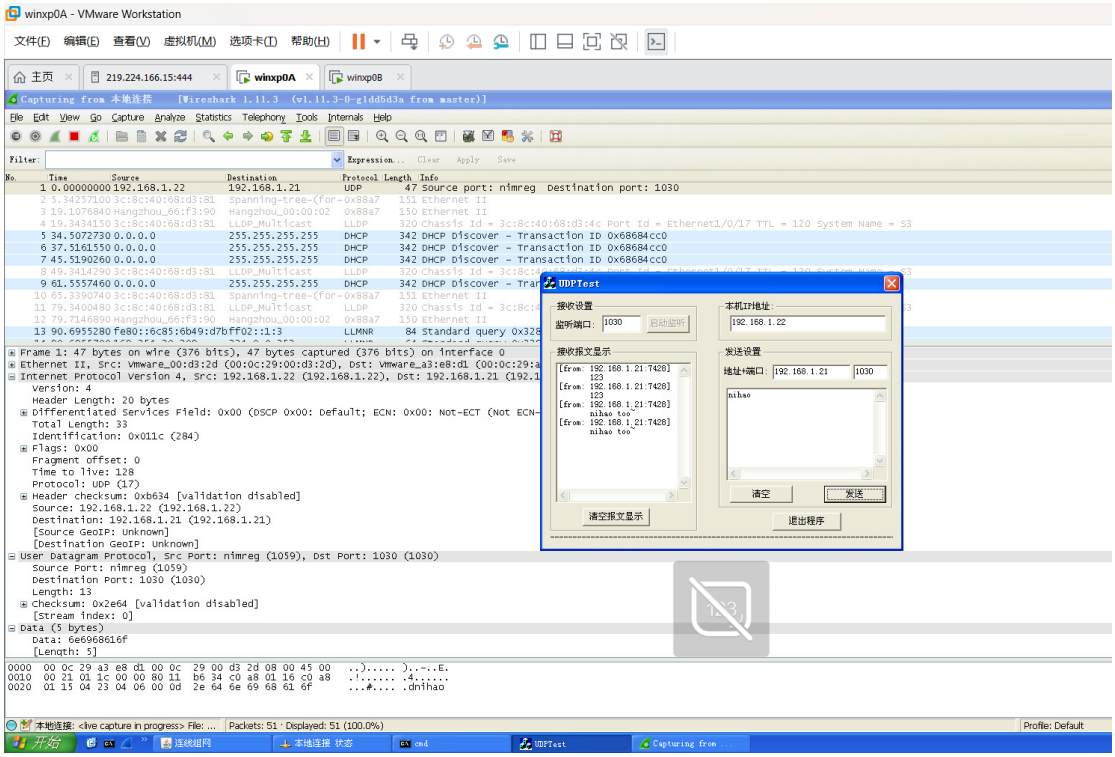
[Router-GigabitEthernet0/0]undo qos lr outbound

[Router-GigabitEthernet0/0]interface GigabitEthernet 0/1

[Router-GigabitEthernet0/1]undo qos lr outbound


linux0C - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
主页
219.224.166.15:444
linux0A
linux0B
linux0C
Applications Places System
root@ubuntu: ~
File Edit View Terminal Help
* Without the owner's prior written consent, *
* no decompiling or reverse-engineering shall be allowed. *
User interface con0 is available.
Please press ENTER.
<r1>
#Jan 3 15:13:08:847 2024 r1 SHELL/4/LOGIN:
Trap 1.3.6.1.4.1.25506.2.2.1.1.3.0.1<hh3cLogIn>: login from Console
%Jan 3 15:13:08:847 2024 r1 SHELL/4/LOGIN: Console login from con0
<r1>
<r1>
<r1>sys
System View: return to User View with Ctrl+Z.
[r1]
[r1]int g0/0
[r1-GigabitEthernet0/0]undo qos lr outbound
[r1-GigabitEthernet0/0]int g 0/1
[r1-GigabitEthernet0/1]undo qos lr outbound
[r1-GigabitEthernet0/1]

11. 根据 4.6 中步骤 7：


winxp0A - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
主页
219.224.166.15:444
winxp0A
winxp0B
UDPTest
接收设置
监听端口: 1030
启动监听
本机IP地址:
192.168.1.22
接收报文显示
发送设置
地址+端口: 192.168.1.21 1030
nihao
清空
发送
清空报文显示
退出程序

（1）分析 UDP 报文结构：选中第一个 UDP 报文，将 UDP 协议树中各字段名、字段长度、字段值、字段表达信息，填入下表。并绘制 UDP 报文结构，详细绘制 UDP 协议树字段。

字段名	字段长度	字段值	字段表达信息
Source Port	2	1059	源端口
Destination Port	2	1030	目的端口
Length	2	13	长度
Checksum	2	0x2e64	校验和
Data	5	6e6968616f	数据

（2）UDP 报文结构与 TCP 报文结构有什么区别？

UDP 报文仅有源端口、目的端口、报文长度、校检码和数据组成。

TCP 除此之外还有 seq、ack、偏移字段等字段，用于保证传输的可靠性。

（3）在步骤 5 交换机 S1 和 S2 之间的网线拔掉期间，PCA 向 PCB 发送的 UDP 消息，在步骤 6 交换机 S1 和 S2 之间的网线重新插上之后，PCB 是否还能收到？请解释为什么会出现这种现象？

PCB 不能收到。因为 UDP 是不可靠传输协议，因此因为拔掉网线而发送失败后报文就丢失了。

（4）综合分析 TCP 协议和 UDP 协议的不同之处。

连接特性：

UDP 是无连接的，不需要在传输数据之前建立连接。

TCP 是面向连接的，需要在数据传输之前建立连接，并在传输结束后释放连接。

传输特性：

UDP 提供不可靠传输，不保证数据的可靠性和顺序性。

TCP 提供可靠传输，确保数据的可靠性和顺序性，同时进行错误检测和重传。

通信模式：

UDP 支持一对一，多对多，多对一，多对多通信。

TCP 仅支持一对一通信，即每个 TCP 连接只能有一个发送方和一个接收方。

数据处理方式：

UDP 是面向数据包（数据报）的，即面向报文。
TCP 是面向字节流的，它将数据视为一连续的字节流，而不是分割成数据包。
头部长度：
UDP 的头部固定为 8 字节。
TCP 的头部长度可变，通常在 20 至 60 字节之间，根据选项字段的使用情况而变化。
综合来看，UDP 适合那些对数据传输实时性要求较高、可以容忍少量数据丢失的应用场景，而 TCP 适合对数据可靠性和顺序性要求较高的场景，如文件传输、网页浏览等。